Week 6: Airflow Overview

Keeyong Han

Table Of Contents

1. Recap of the 5th Week
2. Quick Docker Introduction
3. Overview of Airflow
4. Airflow Code Structure (“Hello World”)
5. Porting Country/City Pipeline to Airflow
6. Break
7. SQL JOIN (LEFT and FULL) - Optional (Week 7)
8. Running Forecasting Job as Airflow Job
9. Demo & Homework #5
10. More on Lab #1
11. Group Project & Mid-Term

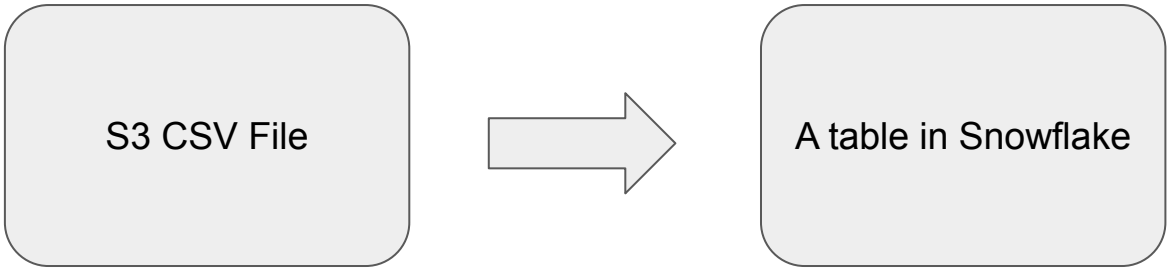
Recap of the 5th Week

Key Concepts

- Full Refresh, Incremental Update
- Idempotency
- SQL Transaction along with try/except
- Backfill
  - Rerunning a data pipeline to fill incomplete or missing data
  - This makes data engineer’s job harder
- Upsert
  - Snowflake supports an operation called MERGE
  - Update existing records and insert new records
    - Most efficient way of ensuring the primary key uniqueness

First Data Pipeline Implementation

- [Data Pipeline Example from Google Colab]



country	capital
Abkhazia	Sukhumi
Afghanistan	Kabul
Albania	Tirana
Algeria	Algiers
American Samoa	Pago Pago

```
create table country_capital (
    country varchar primary key,
    capital varchar
);
```

Concepts of Transaction along with try/except

```
try:
    cur.execute("BEGIN;")
    # execute SQLs that need to be run atomically
    …
    cur.execute("COMMIT;")
except Exception as e:
    cur.execute("ROLLBACK;")
    raise
```

Full Refresh Implementation

```
try:
    cur.execute("BEGIN;")
    cur.execute(f"DELETE FROM {target_table}")
    cur.execute(f"COPY INTO {target_table} …")
    cur.execute("COMMIT;")
except Exception as e:
    cur.execute("ROLLBACK;")
    raise
```

Full Refresh Implementation

```
try:
    cur.execute("BEGIN;")
    cur.execute(f"CREATE TABLE IF NOT EXISTS {target_table} ( …")
    cur.execute(f"DELETE FROM {target_table}")
    cur.execute(f"COPY INTO {target_table} …")
    cur.execute("COMMIT;")
except Exception as e:
    cur.execute("ROLLBACK;")
    raise
```

Snowflake does not support DDL operations within a transaction and always commits immediately

Quick Docker Introduction

What if my program doesn’t run properly in other’s computer

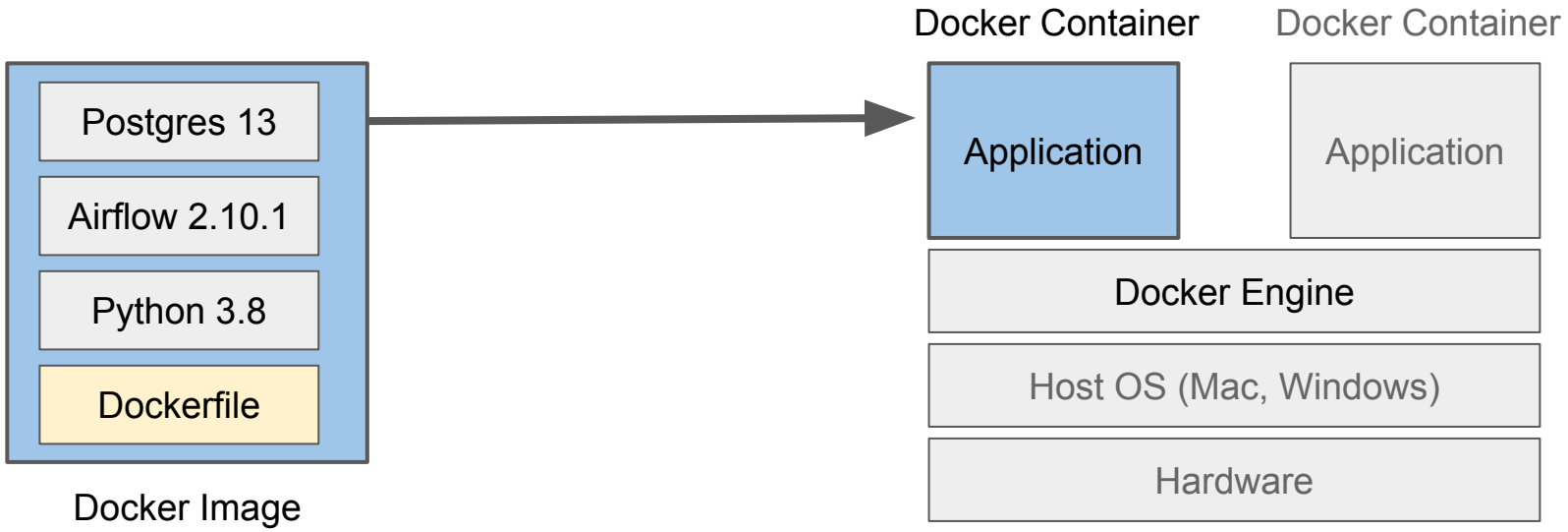
- Different OSs
- Mismatched versions of libraries
- Missing important files during the installation process

Docker to the Rescue

Virtual Linux computers within your computer

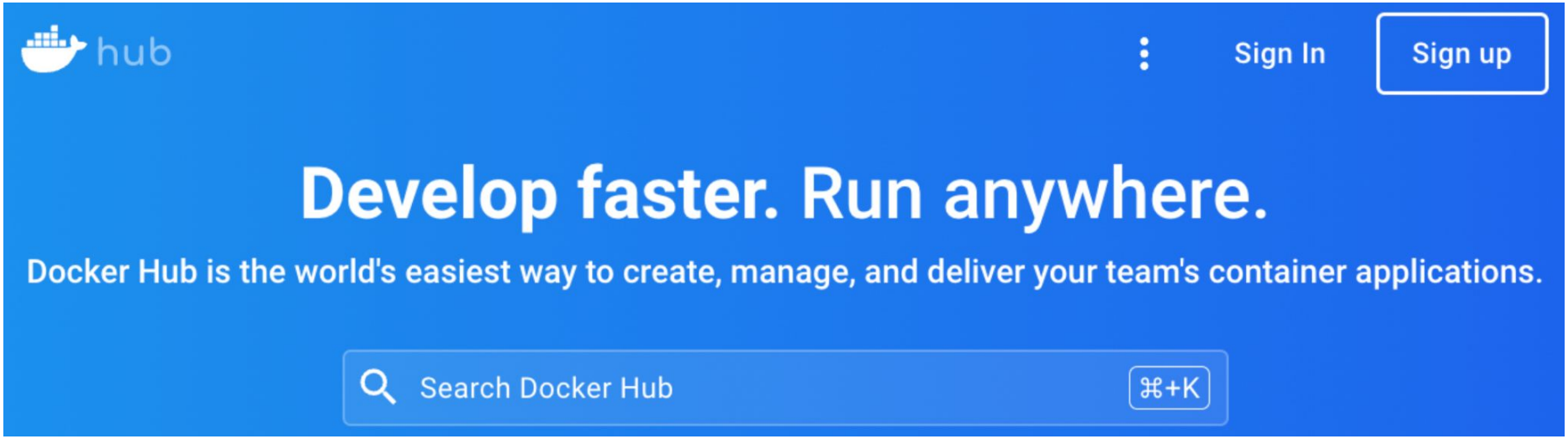
What if you could package your computer environment exactly give it to someone else?

- Docker Image: A independent and complete software package
- Docker Container: An isolated environment running a docker image



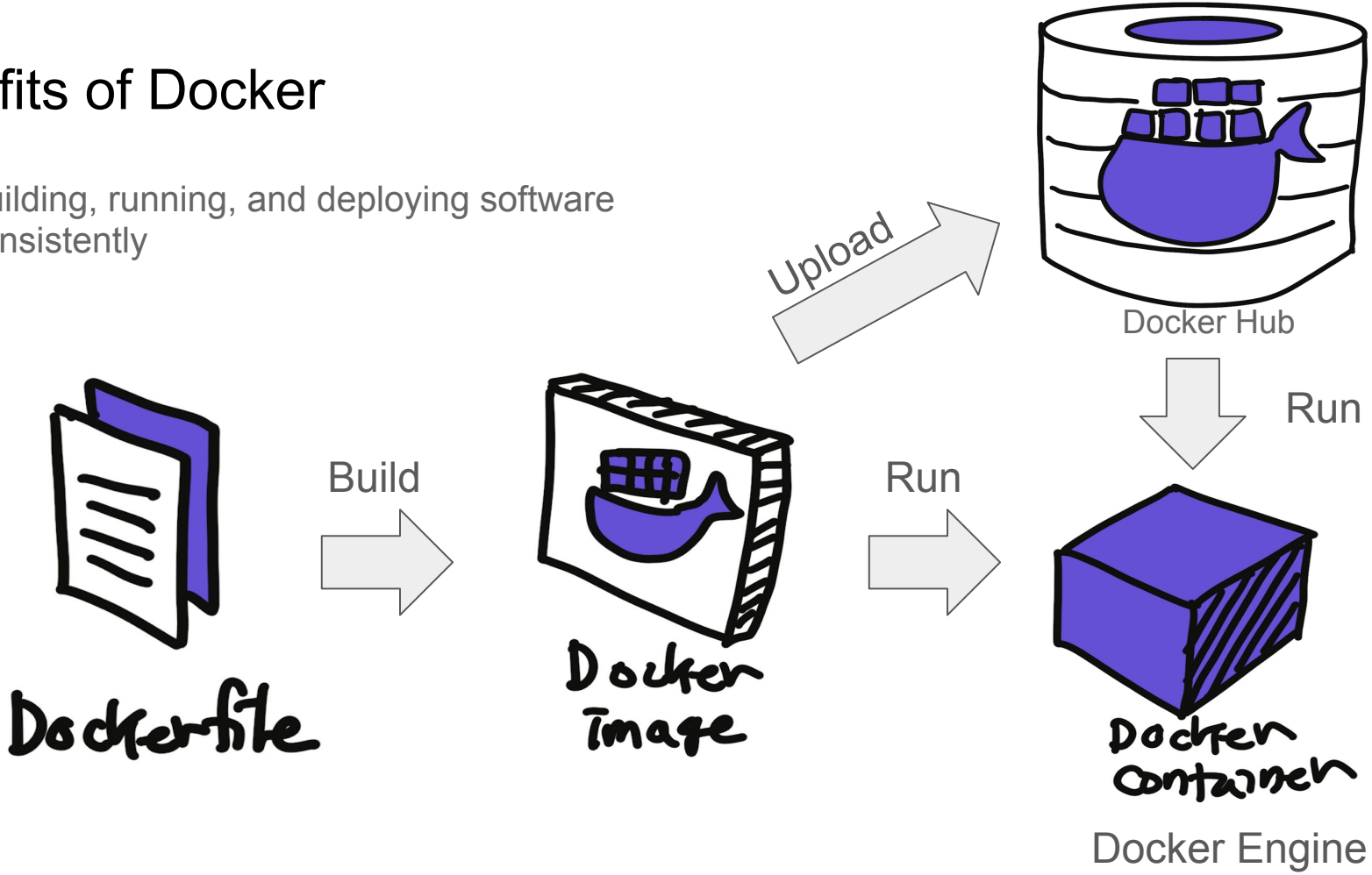
How to Share Docker Images with Others

- Docker Registry Service: A place to share Docker Images
- Docker Hub is the most well-known Docker registry



Benefits of Docker

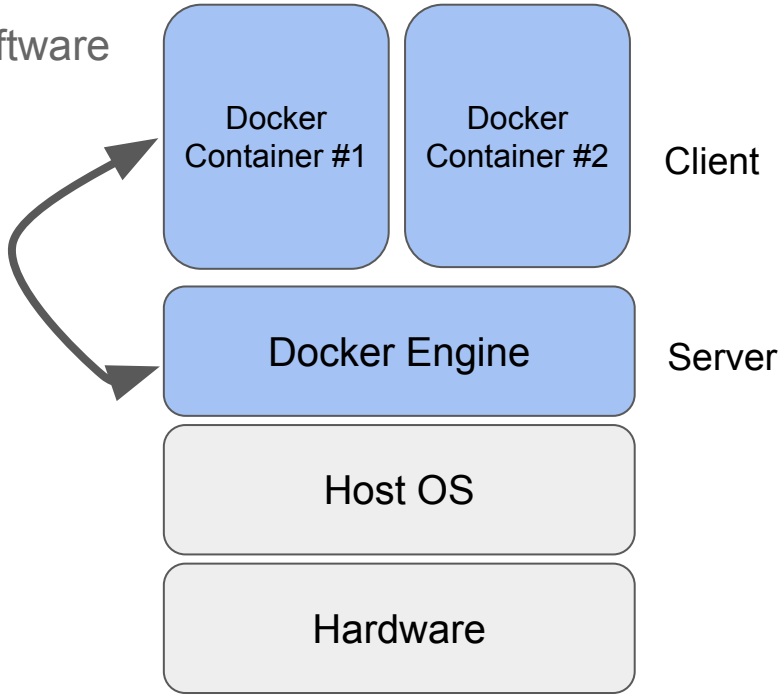
- Building, running, and deploying software consistently

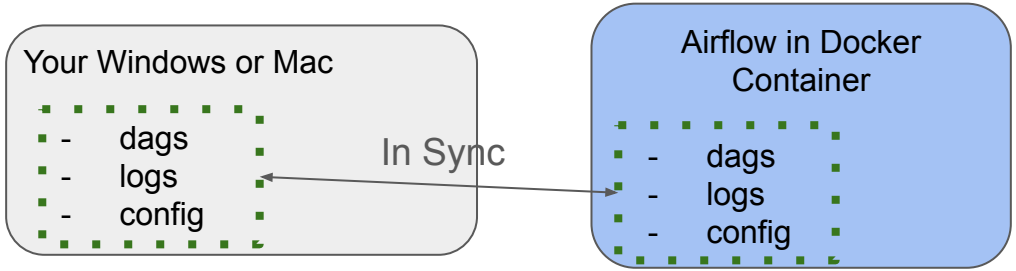


What is Docker Container?

- An independent and isolated space for running software
- It has its own file system (called a Volume)
  - It can be mirrored from host OS

Host OS	Supported Container OS
Mac	Lightweight **Linux** VM
Windows	Windows, **Linux**
Linux	**Linux**





Overview of Airflow

What is Airflow? (1)

- Airflow is a data pipeline framework written in Python
  - An Apache open-source project that started at Airbnb
  - The most widely used framework for managing/writing data pipelines
  - For batch data processing (not ~~for real-time data processing~~)

Company	Big Data System	Data Pipeline framework
Apple	Spark + Iceberg	Airflow
Pinterest	Spark	Airflow
PepsiCo	Spark (Databricks), Snowflake	Airflow + dbt
Udemy	Spark (Databricks) /Hive/Redshift	Airflow + dbt
Uber	Hive/Presto/Spark	Airflow
Airbnb	Hive/Presto	Airflow

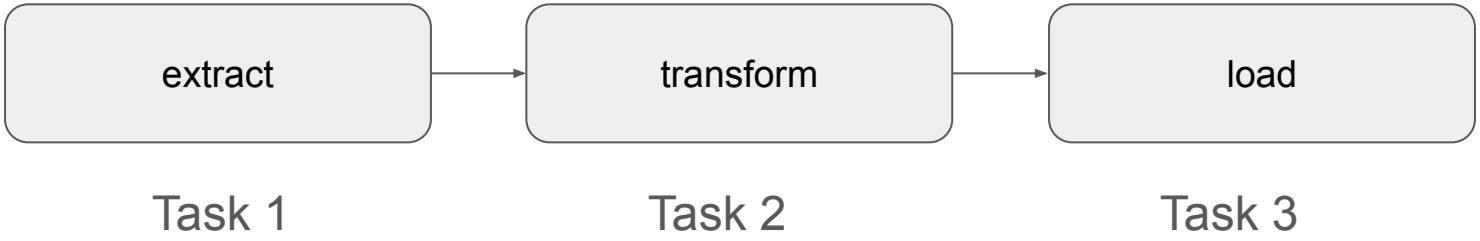
Airflow

What is Airflow? (2)

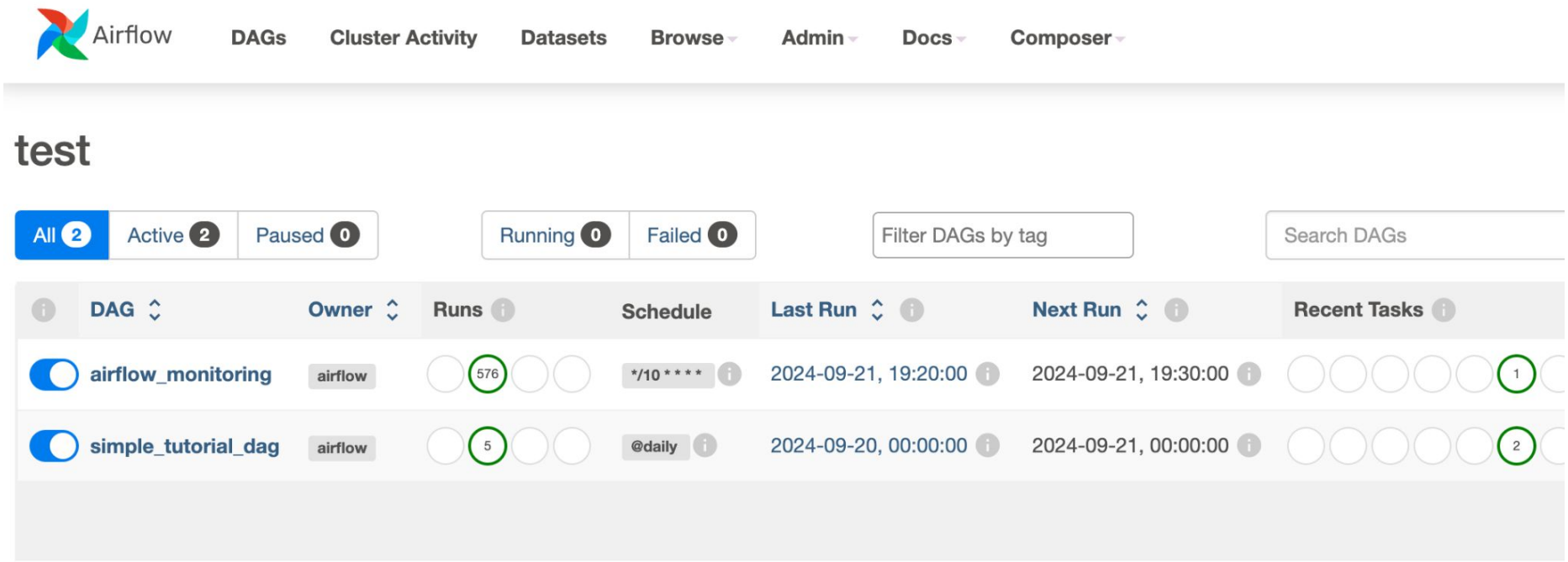
- Supports data pipeline scheduling
  - Executes at set times or triggers the next pipeline when one pipeline's execution ends
  - Also provides triggering through web UI, command line, or API
- Makes writing and operating data pipelines easier
  - Provides Python modules that easily integrate various data sources and data warehouses
  - Offers various features related to data pipeline management: especially "**Backfill**”

What is Airflow? (3)

- In Airflow, data pipelines are called DAGs (Directed Acyclic Graphs)
  - A single DAG consists of one or more tasks
  - The example below is a case where a DAG is composed of 3 tasks
- A python function can be turned into a task (will explain later)



Airflow Web UI



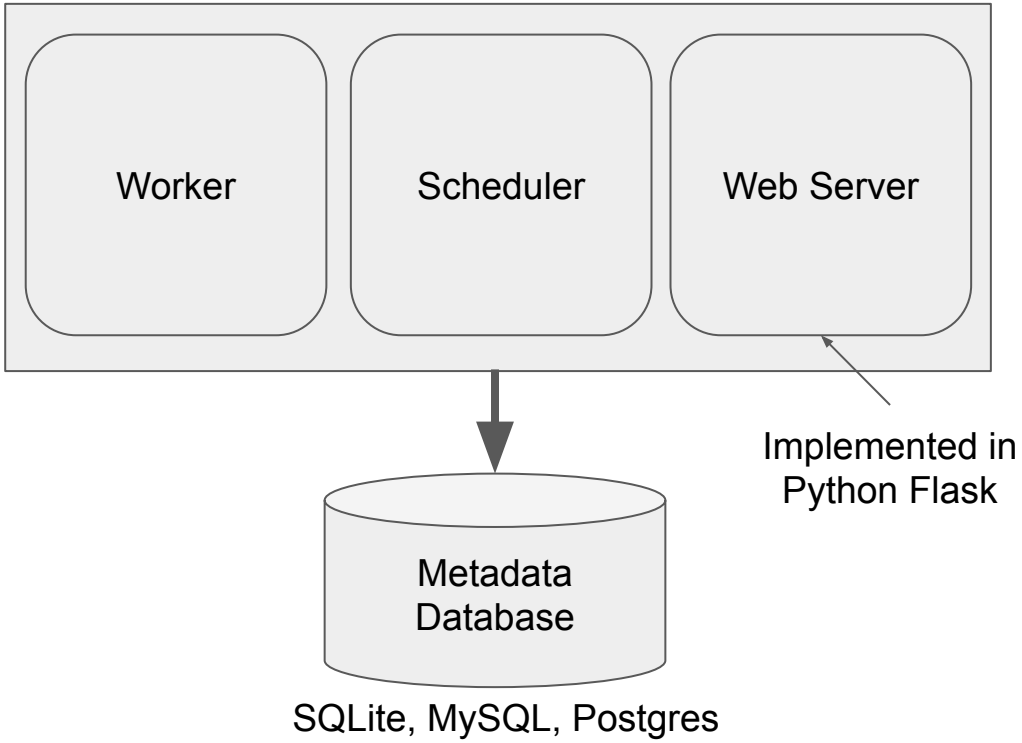
Pros and Cons of Airflow

- Advantages
  - Allows fine-grained control of data pipelines
  - Supports various data sources and data warehouses
  - (If properly written) Backfilling is a lot easier
- Disadvantages
  - Not easy to learn (somewhat steep learning curve in particular for incremental update)
  - Relatively difficult to set up development/testing environments
  - Not easy to operate directly. From multi-server versions, cloud versions are preferred
    - **GCP: “Cloud Composer”**
    - AWS: “Managed Workflows for Apache Airflow”
    - Azure: “Data Factory Managed Airflow”

Airflow - Composed of 6 components

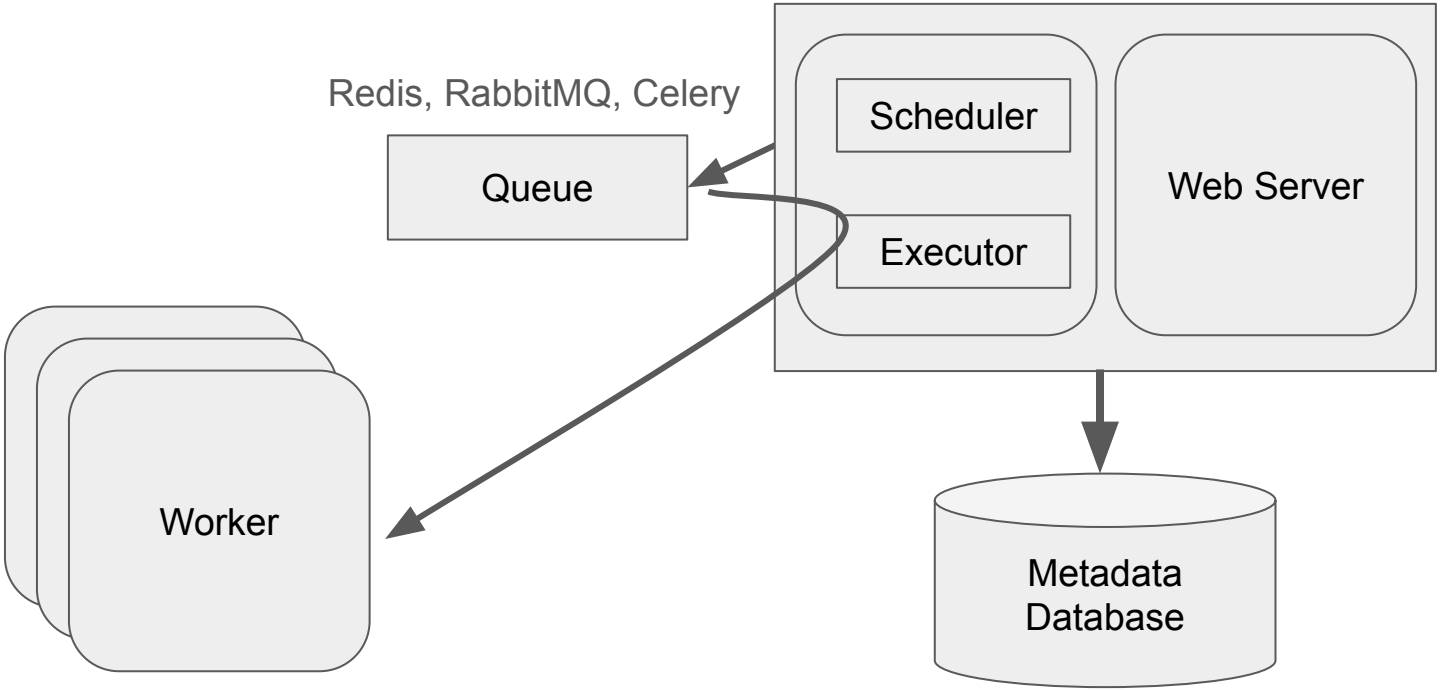
1. Web Server
   a. The web UI visualizes the execution status of the scheduler and DAGs
2. Scheduler
   a. The scheduler is responsible for assigning tasks to workers
3. Worker(s)
   a. Workers perform the actual execution of DAGs
4. Triggerer
   a. Responsible for running deferred tasks (usually some external conditions related)
5. Meta Database
   a. Sqlite is used by default. In actual production, MySQL or Postgres should be used
   b. The execution results of the scheduler and each DAG are stored in a separate database
6. Task Queue (Only used for multi-server configuration)

Airflow Configuration: A Single Server



Scale Up
Scale Out
Use Cloud Service

Airflow Configuration: Multi Workers

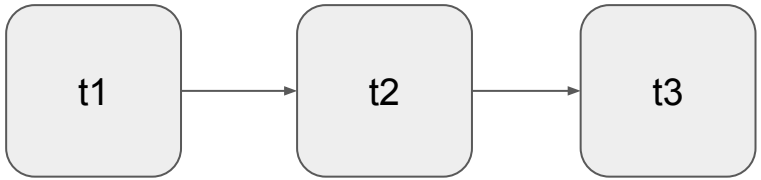


Airflow Code Structure

What is DAG?

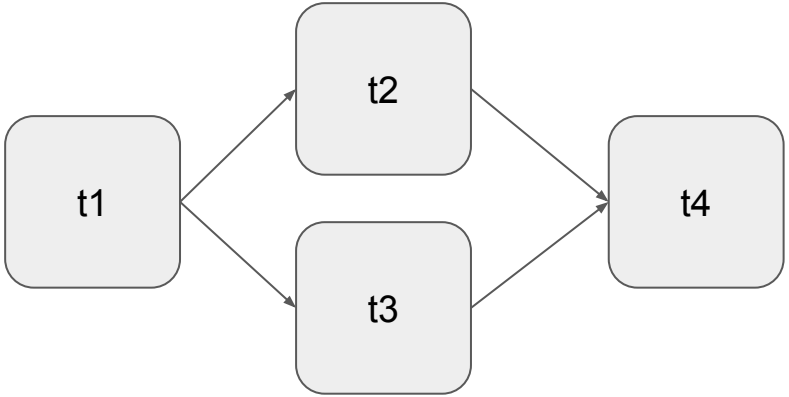
- Directed Acyclic Graph
  - DAG is composed of tasks
- What is a task?
  - Created using Airflow Operators
    - ex) **SnowflakeOperator**, MySqlOperator, BashOperator, …
    - ex) task decorator (we will use this one mostly)
  - Most versatile one is PythonOperator (running a Python function as a task)
  - Airflow already provides various operators out of the box
    1. Choose the appropriate operator for the situation or develop one directly if needed
    2. e.g., Snowflake, Redshift, Postgres, S3, Spark job, Python, …
- We will be mostly using Python Operator (decoration syntax)

DAG Example (1)



- DAG composed of 3 tasks
- Tasks are running sequentially

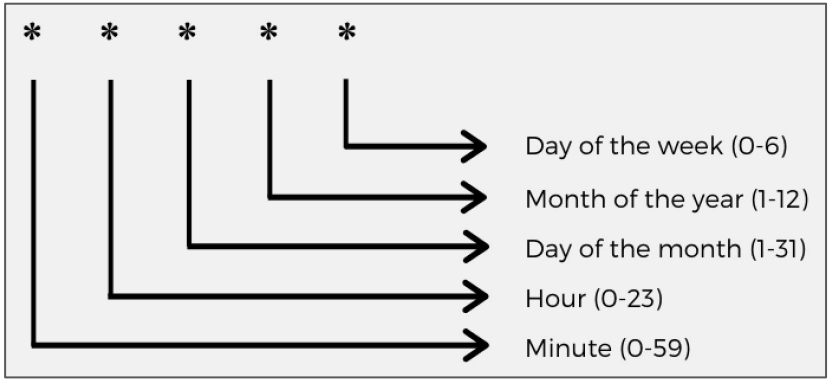
DAG Example (2)



- DAG composed of 4 tasks
  - First, t1 is executed
  - From there it branches into t2 and t3
  - When t2 and t3 are done, t4 runs

DAG Implementation: Create an instance of DAG

```
from airflow import DAG

test_dag = DAG(
   "dag_v1", # DAG name
   schedule="0 9 * * *",    # follows crontab syntax
   tags=['test'],
   start_date=datetime(2025, 9, 29, hour=0, minute=00),
   default_args=default_args   # will explain this in the next slide
)
```



“0 * * * *” ?

“30 12 * * *” ?

DAG Implementation: Default Argument

```
# Basic information necessary for all tasks
default_args = {
    'owner': 'keeyong',
    'email': ['keeyong.han@sjsu.edu'],
    'retries': 1,
    'retry_delay': timedelta(minutes=3),
}
```

Task Implementation

```
# You can use different Operators
# If Python functions are tasks, use @task decorators
@task
def print_hello():
    print("hello!")
    return "hello!"
```

Task Execution Order

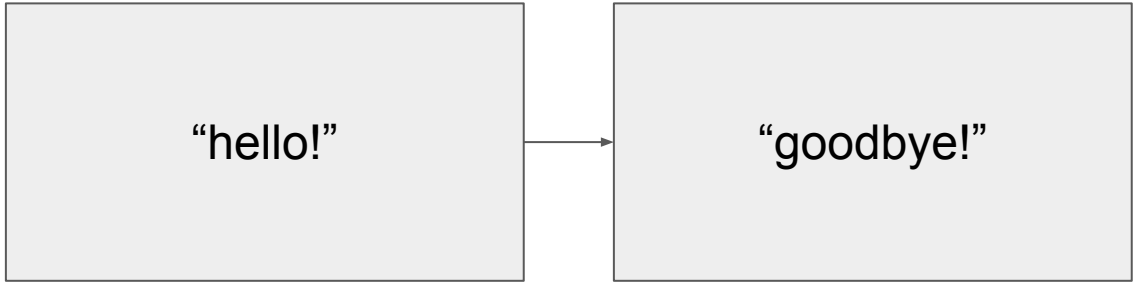
```
# If you have three tasks (or operators) t1, t2, t3

# To run them sequentially
t1 >> t2 >> t3

# To run t1 & t2 in parallel and then t3
t1 >> t3
t2 >> t3    -> [t1, t2] >> t3

# To run t1, and then t2 and t3 in parallel
t1 >> t2
t1 >> t3    -> t1 >> [t2, t3]
```

First Airflow DAG

Airflow @task decorators: marking a function as a task

```
from airflow.decorators import task
from datetime import timedelta
from airflow import DAG
from datetime import datetime

@task
def print_hello():
    print("hello!")
    return "hello!"

@task
def print_goodbye():
    print("goodbye!")
    return "goodbye!"
```

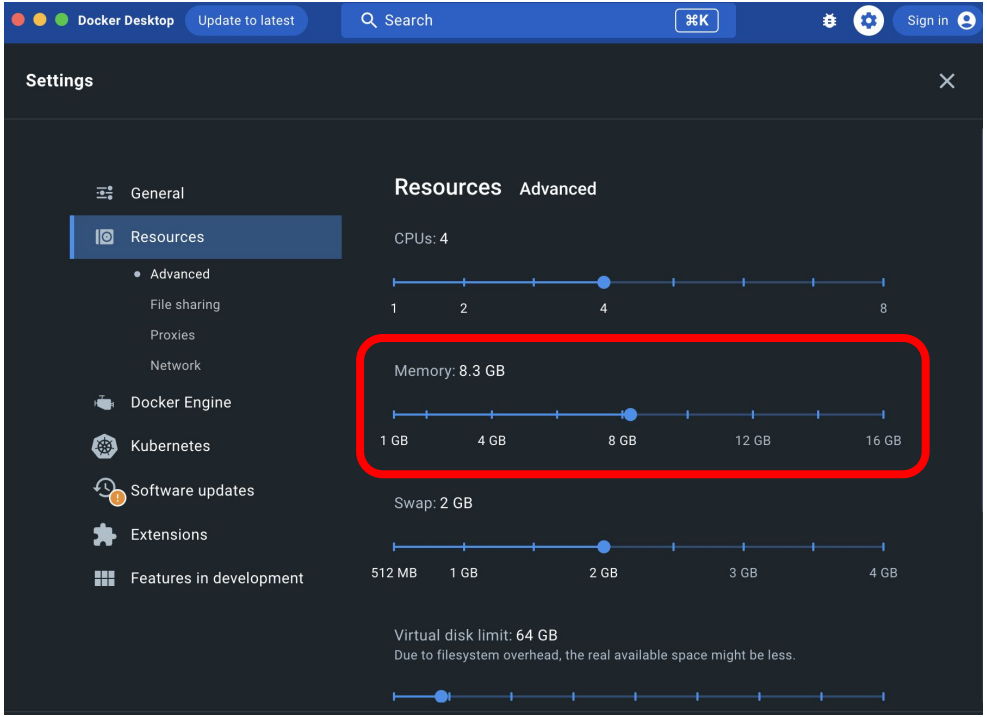
```
with DAG(
    dag_id = 'HelloWorld',
    start_date = datetime(2026,2,24),
    catchup=False,
    tags=['example'],
    schedule = '0 2 * * *'
) as dag:

    # run two tasks in sequence
    print_hello() >> print_goodbye()
```



Airflow Installation (Docker)

Run Docker Engine and check resource allocation

- The memory requirements
  - At least 4GB in Mac
  - At least 6GB in Windows



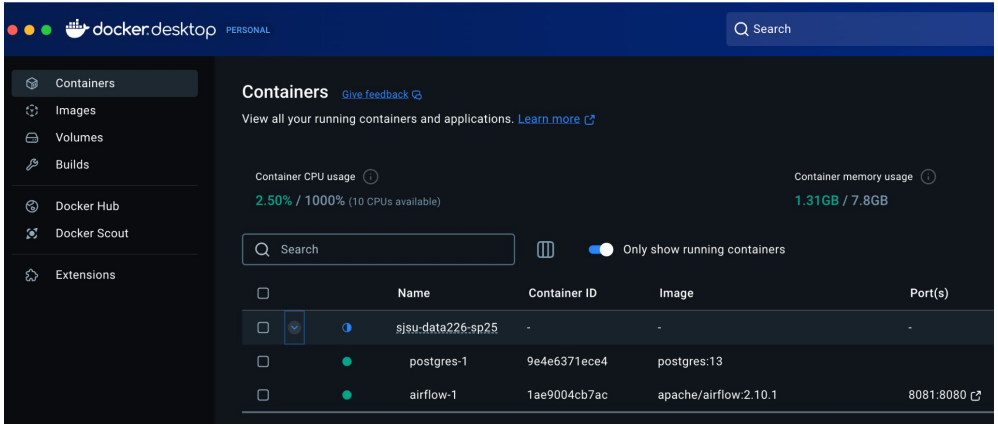
Run Airflow in the Docker Containers #1

- First run your terminal and move to a folder of your choice
- Download data226-SP26 repo
  - `git clone https://github.com/keeyong/sjsu-data226-SP26.git`
  - If you don't have git, please install git first
- Change current directory to sjsu-data226-SP26 folder
  - `cd sjsu-data226-SP26`
  - Here you will see a few sub-folders
    - dags
    - logs
    - plugins
    - config

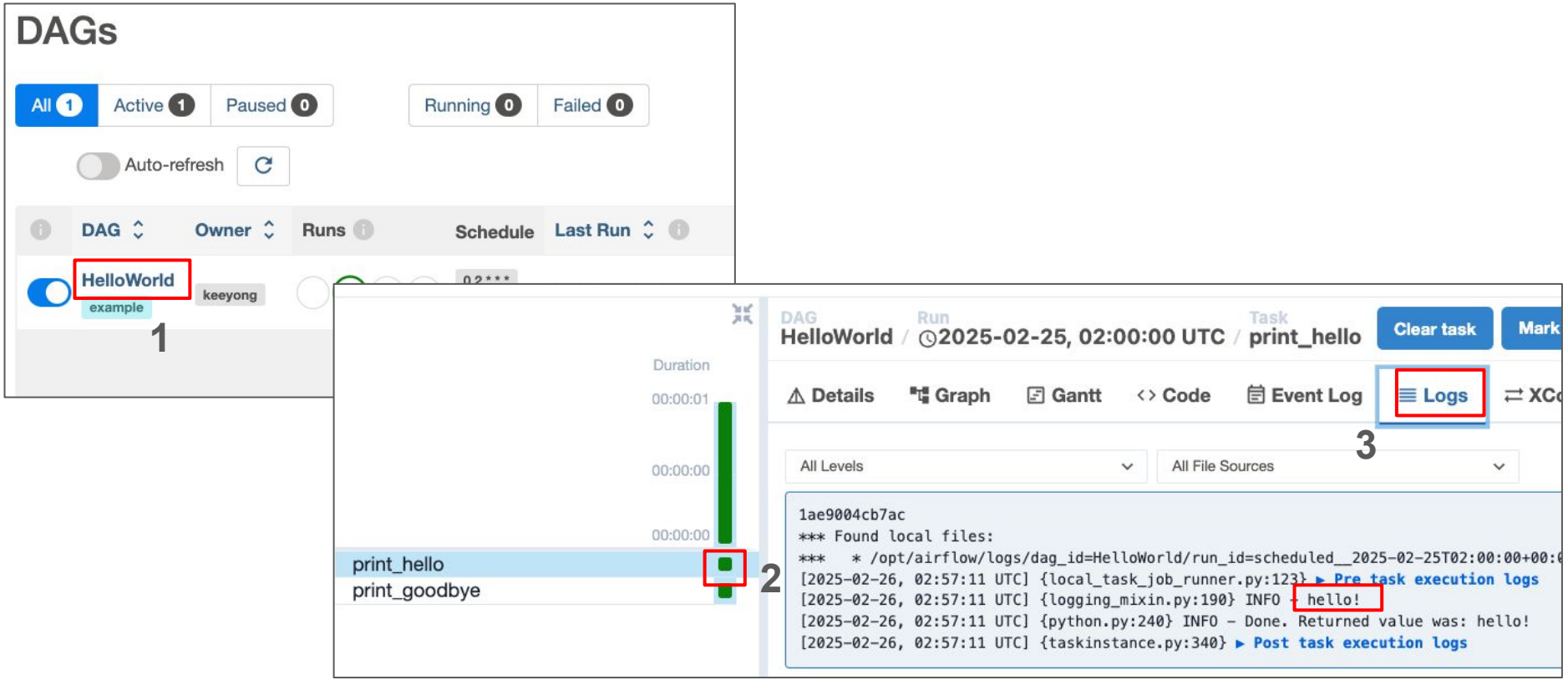
    These folder will be synced with Docker Container filesystem (/opt/airflow/)
  - You should see a file named “docker-compose.yaml” (will check some content together)
    - _PIP_ADDITIONAL_REQUIREMENTS

Run Airflow in the Docker Containers #2

- First initialize Airflow environment
  - `docker compose -f docker-compose.yaml up airflow-init`
- Next run the Airflow service
  - `docker compose -f docker-compose.yaml up`
- Visit http://localhost:8081 and log in with airflow:airflow as your ID:PW
  - Set up Variables and Connections as necessary



Checking HelloWorld DAG from UI



Log in to Airflow Docker Container

- Open a terminal
- Run “docker ps” command to get the ID of Airflow Docker Container

```
keeyong sjsu-data226-SP26 % docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED       STATUS
PORTS                    NAMES
1ae9004cb7ac   apache/airflow:2.10.1   "/usr/bin/dumb-init …"   5 hours ago   Up 5
hours (healthy)   0.0.0.0:8081->8080/tcp   sjsu-data226-sp26-airflow-1
9e4e6371ece4   postgres:13            "docker-entrypoint.s…"   5 hours ago   Up 5 hours
(healthy)   5432/tcp                 sjsu-data226-sp26-postgres-1
```

- Run “docker exec” command to log in to the container

```
keeyong sjsu-data226-SP26 % docker exec -it sjsu-data226-sp26-airflow-1 sh
(airflow)
```

Airflow Commands

- airflow dags list
- airflow tasks list HelloWorld
- airflow dags test HelloWorld

(airflow) airflow dags list

dag_id | fileloc | owners | is_paused

==========+==========+=========+==========

HelloWorld | /opt/airflow/dags/hello_world.py | keeyong | False

(airflow) airflow tasks list HelloWorld

print_goodbye

print_hello

(airflow) airflow dags test HelloWorld

[2025-02-26T04:31:17.793+0000] {dagbag.py:587} INFO - Filling up the DagBag from **/opt/airflow/dags**

[2025-02-26T04:31:17.821+0000] {dag.py:4411} INFO - dagrun id: **HelloWorld**

Porting Country/City Pipeline to Airflow

What’s the benefit of porting to Airflow

- You can schedule it or define trigger dependency
- You can check the status/log from its Web UI
- You can set up alerts in case it doesn’t execute as expected
- You can backfill easily for incremental update
- You can hide credentials in more secure fashion
  - Using Airflow Variables and Connections

Import Airflow related modules

```
from airflow import DAG
from airflow.models import Variable
from airflow.decorators import task

from datetime import timedelta
from datetime import datetime
import snowflake.connector
import requests
```

How to add Snowflake package to Airflow Docker Container?

- snowflake-connector-python module is needed!
- Modify _PIP_ADDITIONAL_REQUIREMENTS of docker-compose.yaml file
  - If you add any new python module(s), need to restart the docker containers
    - docker compose -f docker-compose.yaml **down**
    - docker compose -f docker-compose.yaml **up**

```
…
  _PIP_ADDITIONAL_REQUIREMENTS: ${_PIP_ADDITIONAL_REQUIREMENTS:- yfinance apache-airflow-providers-snowflake snowflake-connector-python}
…
```

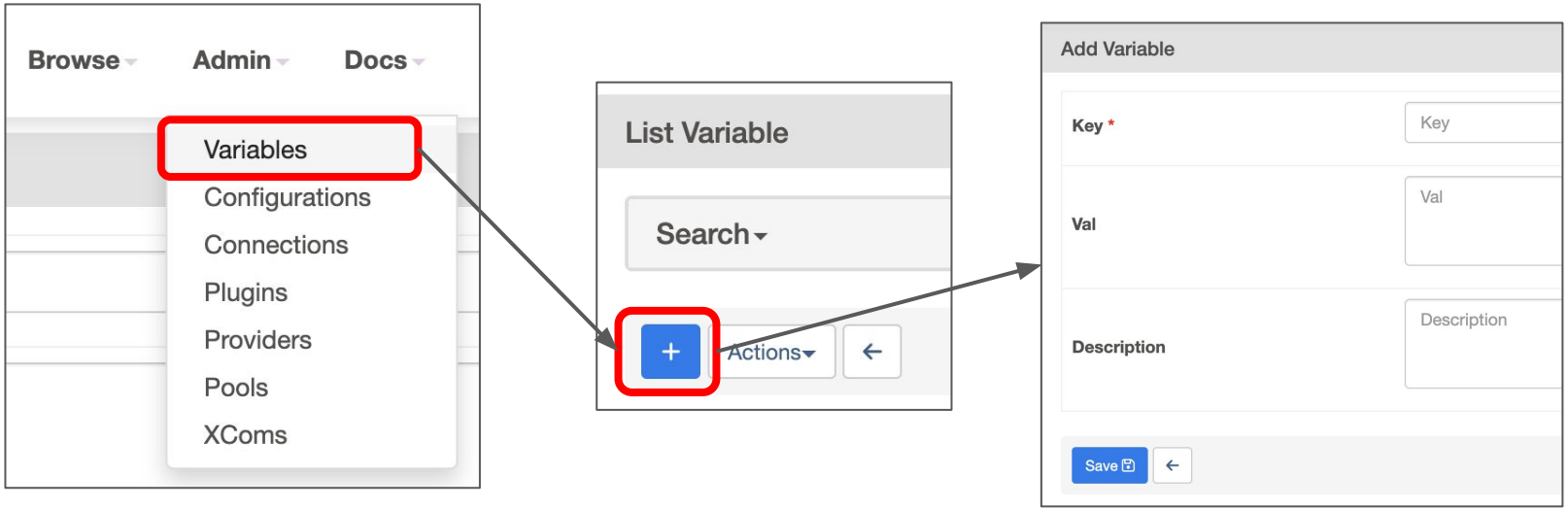
Let’s take a look at [docker-compose.yaml]() file together

Airflow Variables (1)

- A way to store and retrieve small key-value pairs
  - Make your DAGs more dynamic by adjusting settings without changing the code
- Stored in the Airflow metadata database
  - can be managed via the UI, API, or command-line interface.
- Use Variable.get to read or Variable.set to write
- Can mask the values (as “****”)
  - If the variable name contains of the followings:
    - access_token, api_key, apikey, authorization, passphrase, passwd, password, private_key, secret, token

Airflow Variables (2)

- Open Web UI, select Admin ➔ Variables and add the followings
  - country_capital_url: https://s3-geospatial.s3.us-west-2.amazonaws.com/country_capital.csv
  - snowflake_userid, snowflake_password, snowflake_account, snowflake_database, snowflake_warehouse



Review the entire code

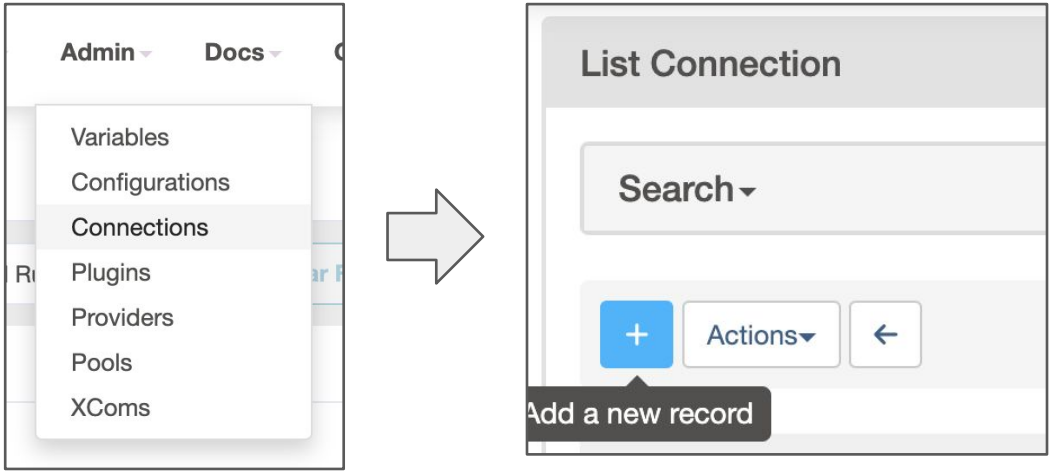
- GitHub link
- Let’s improve this by using **Snowflake Connection**

Airflow Connections (1)

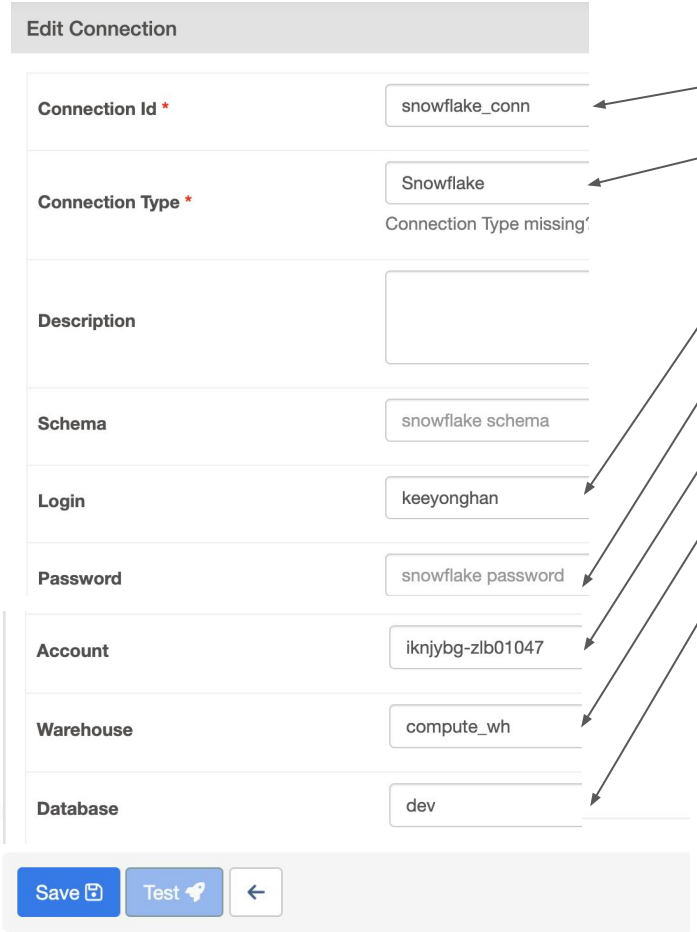
- A way to make connections with external systems more secure
  - You don’t have to expose connection related credentials in your code
  - Can be used to connect to external systems such as databases, cloud storage, or APIs
- Stored in the Airflow metadata database
  - can be managed via the UI, API, or command-line interface
- Easier to manage than using a bunch of Airflow variables

Airflow Connections (2)

- Set up a connection called “snowflake_con” from Admin -> Connections



Airflow Connections (2)



- Connection Id: snowflake_conn
- Connection Type: snowflake
- Login: <SNOWFLAKE ID>
- Password: <SNOWFLAKE PASSWORD>
- Account: <SNOWFLAKE ACCOUNT>
- Warehouse: <YOUR SNOWFLAKE WH>
- Database: <YOUR SNOWFLAKE DB>

Schema, Warehouse, Database are optional as long as you set it in your code
Account should be “sfedu02-eab27764”

Review the v2 code and Run

- Code Review: GitHub link
- How to run this DAG?
  - Move the DAG file to dags folder
  - Airflow will scan the dags folder to find new/modified/deleted DAGs every 5 minutes
- Open a terminal and move to sjsu-data226-SP26 folder

```
keeyong sjsu-data226-SP26 % cp week6/country_capital_to_snowflake* dags/
```

- Log in to Airflow docker container
- Run the following commands
  - airflow dags list
  - airflow tasks list CountryCaptial
  - airflow dags test CountryCaptial

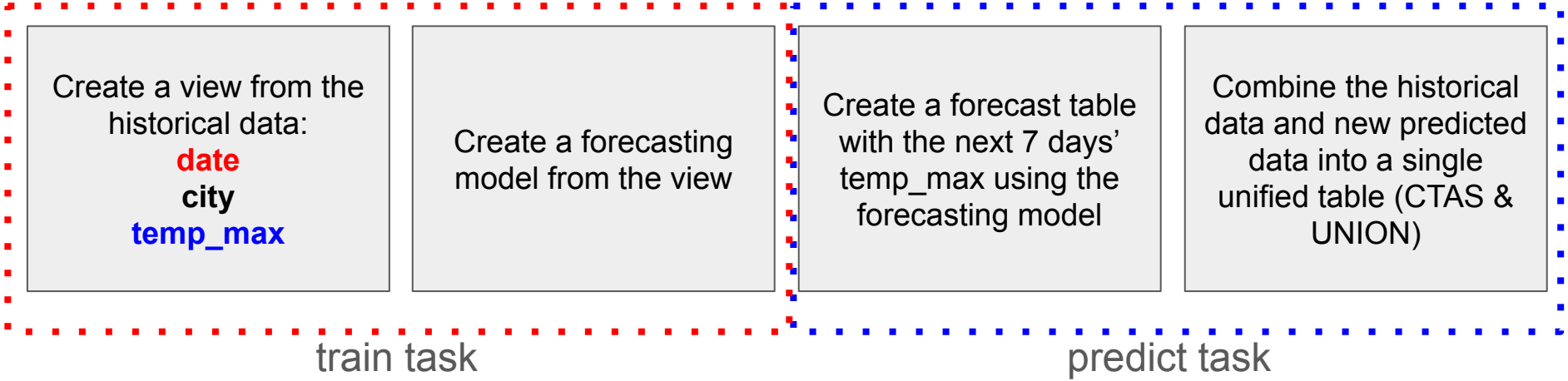
Demo: Running Forecasting Job as Airflow Job

How to Schedule ML jobs?

- There are two options: Pros and cons
  - As an Airflow DAG
  - As a task in Snowflake
- We will use Airflow DAG approach
  - It is easier to manage (in a single place)
  - It is easier to define dependencies with other jobs
  - Especially if you already have Airflow in your production environment

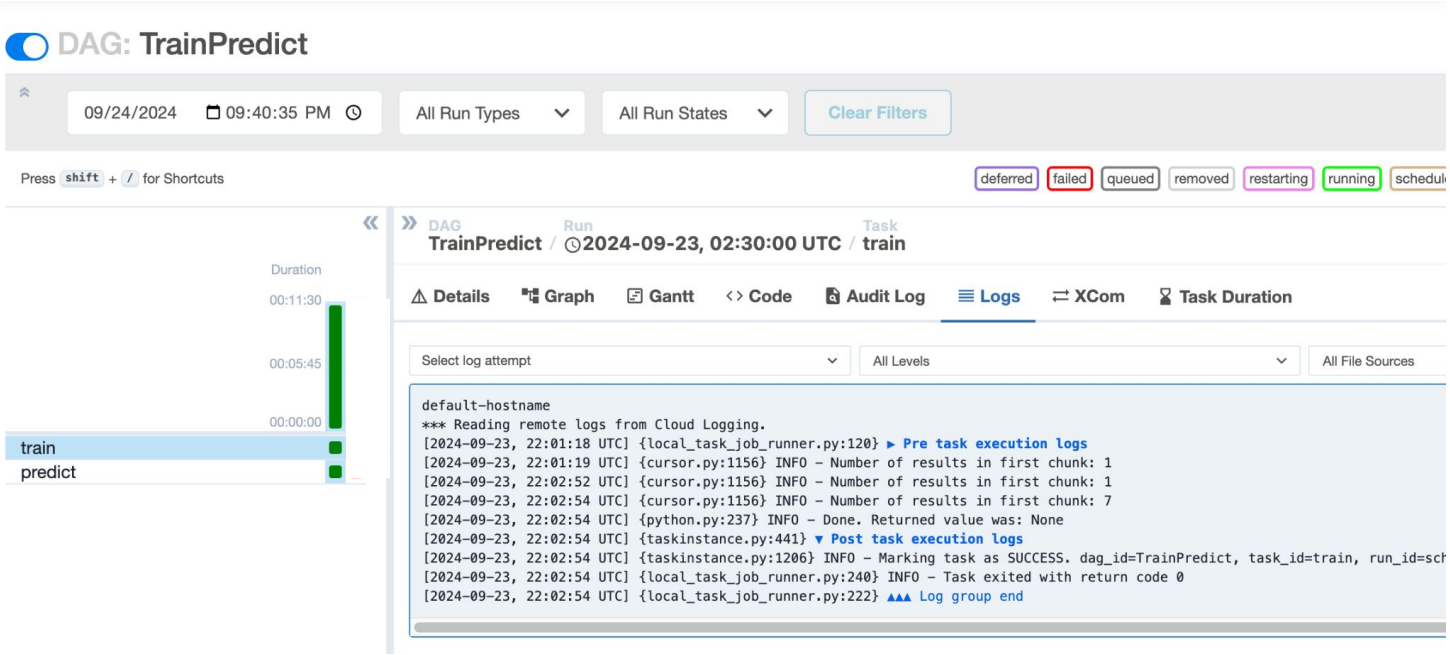
Forecasting stock prices in two tasks

- Let's say we have a table called "dev.raw.weather_data" with past 60 day data
  - Date, latitude, longitude, temp_max, temp_min
- Now we want to predict the next 7 days' weather based on the table
  - Build a model first and then create a table with predicted temperature (next 7 days)
- Create a union table from both tables (60 days + 7 days)



Code Review

- [Github link]

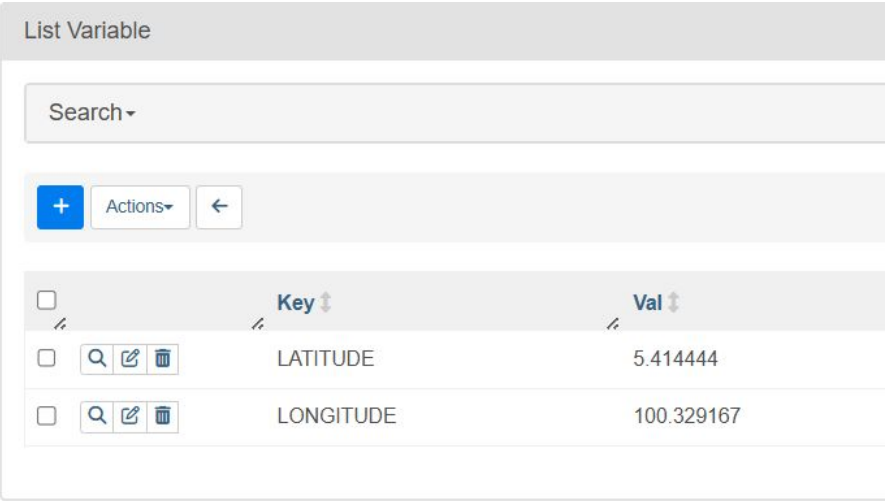


Homework #5

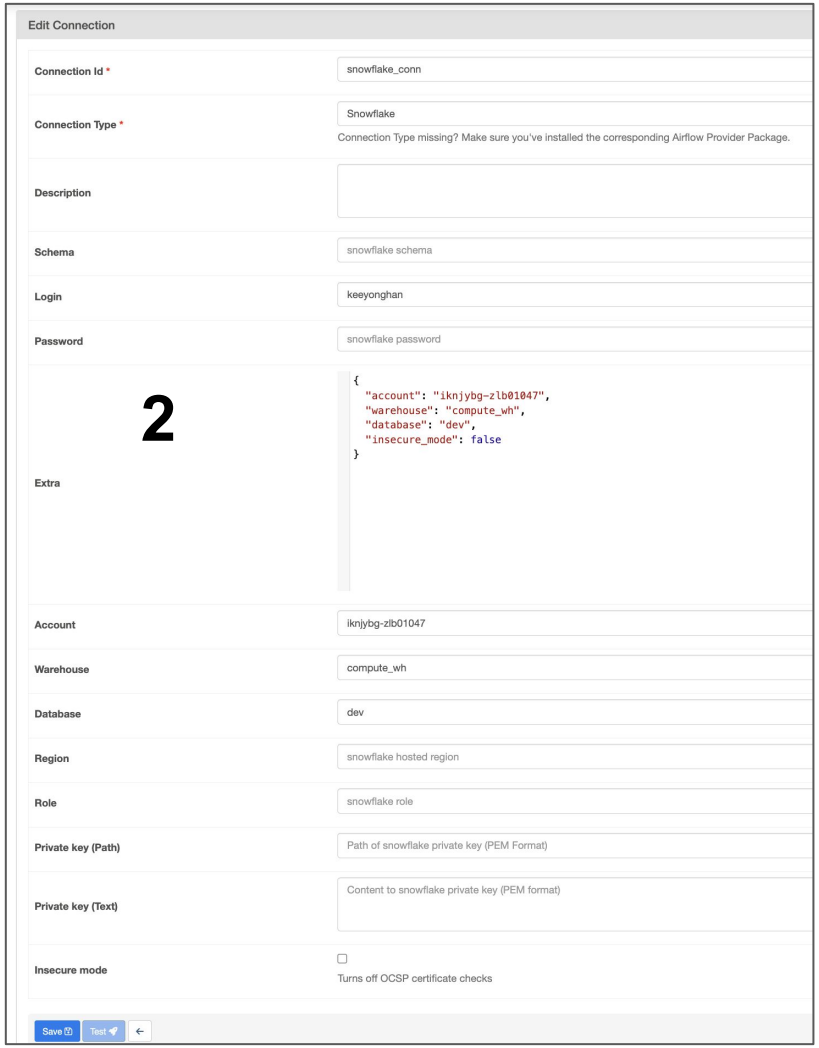
Porting homework #4 to Airflow (13 pts)

- (+2) Create tasks using @task decorator (refer to GitHub link)
  - You can use as many tasks as you want
  - Schedule the tasks properly (task dependency)
- (+1) Set up Airflow variables for latitude and longitude
  - Use the variable in your code (Variable.get)
  - Capture the Admin -> Variables screenshot (an example will be provided ①)
- (+2) Set up Snowflake Connection (refer to GitHub link)
  - Use the connection in your code
  - Capture the Connection detail page screenshot (an example will be provided ②)
- (+5) Ensure the overall DAG is implemented properly and runs successfully
  - A github link with the entire code needs to be submitted (2 pts)
  - Implement the same full refresh using SQL transaction (3 pts)
- (+2) Capture two screenshot of your Airflow Web UI (examples to follow)
  - One with the Airlow homepage showing the DAG (③)
  - The other with the log screen of the DAG (④)
- (+1) Overall formatting

Screenshot Examples (1 & 2)



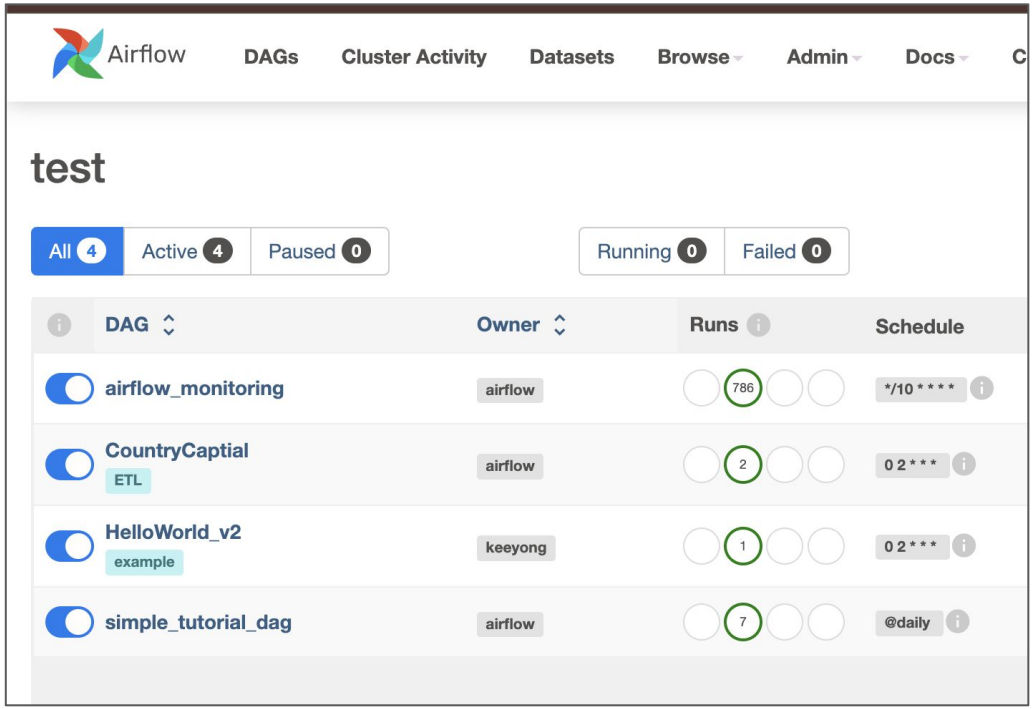
1



2

Screenshot Examples (3): Something like this

3



This is just an example!

Show your homework DAG here!

Screenshot Example (4): Something like this

DAG: CountryCaptial

09/23/2024 06:37:59 AM | All Run Types | All Run States | Clear Filters

Press shift + / for Shortcuts

deferred failed queued remove

DAG CountryCaptial / Run 2024-09-23, 02:00:00 UTC / Task load

5

Duration

00:01:29

00:00:44

00:00:00

extract

transform

load

Details | Graph | Gantt | Code | Audit Log | Logs | XCom | Task Du

(by attempts)

1

All Levels | All File Sources

```
default-hostname
*** Reading remote logs from Cloud Logging.
[2024-09-23, 06:37:05 UTC] {local_task_job_runner.py:120} ▶ Pre task execution logs
[2024-09-23, 06:37:06 UTC] {cursor.py:1156} INFO - Number of results in first chunk: 1
[2024-09-23, 06:37:07 UTC] {cursor.py:1156} INFO - Number of results in first chunk: 1
[2024-09-23, 06:37:07 UTC] {logging_mixin.py:188} INFO - Abkhazia - Sukhumi
[2024-09-23, 06:37:08 UTC] {logging_mixin.py:188} INFO - Afghanistan - Kabul
[2024-09-23, 06:37:08 UTC] {logging_mixin.py:188} INFO - Akrotiri and Dhekelia - Episkopi Cantonment
[2024-09-23, 06:37:09 UTC] {logging_mixin.py:188} INFO - Albania - Tirana
[2024-09-23, 06:37:10 UTC] {logging_mixin.py:188} INFO - Algeria - Algiers
[2024-09-23, 06:37:11 UTC] {logging_mixin.py:188} INFO - American Samoa - Pago Pago
```

This is just an example!

Show your homework DAG here!

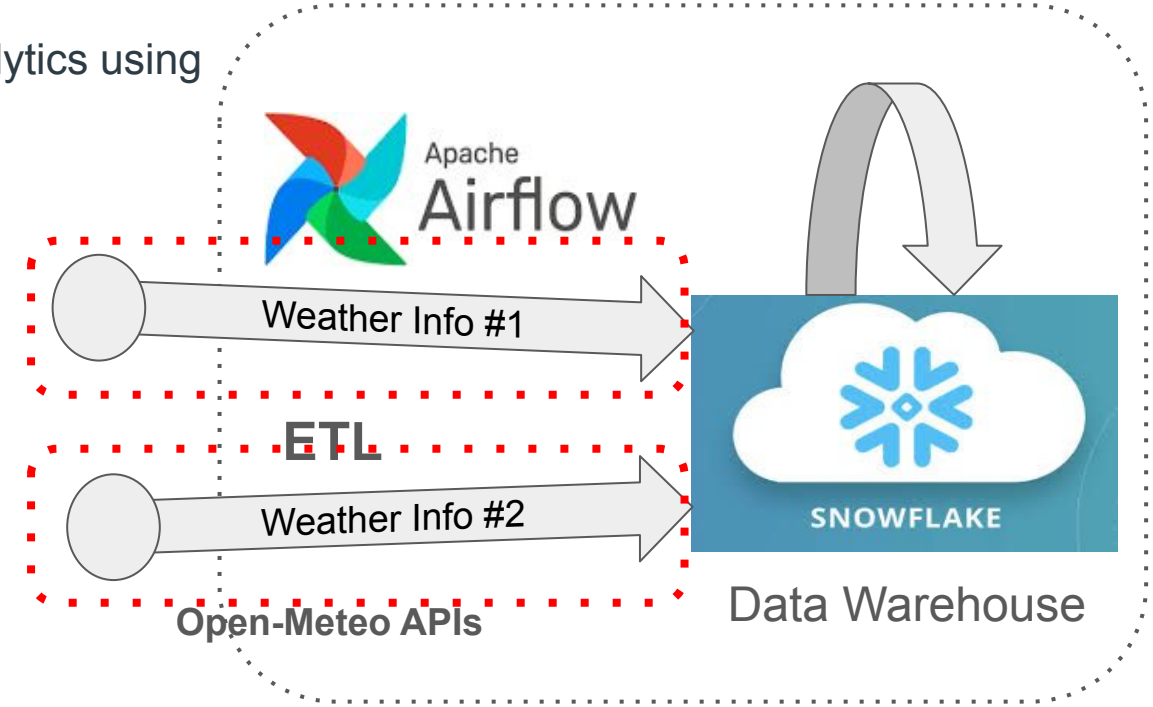
PSA: lab, mid-term, quiz #2, group project and so on

Lab #1

- Due Date: March 5th

Task to forecast the next 7 days of weather (**ELT**)

Building a Weather Prediction Analytics using Snowflake & Airflow



Report Format

- Team Introduction
- Problem Statement
- Dataset(s)
- System Diagram
- Tables
- Implementation Details
- Lessons
- Future Work
- Conclusion

Use screenshots as many as possible
Submit the GitHub link of the project
- GitHub repos should have README.md

What will be in the mid-term exam? (3/26)

- Will cover from week 1 to week 9
- Part A:
  - 50 minutes & 30 questions (60 pts)
  - Multi-choice questions
- Part B:
  - 65 minutes & 10 questions (85 pts)
  - 3 SQL questions (1 of them will be about JOIN, 2 of them will be about SELECT)
  - Python & Concept assessment
    - Short-answer & long-answer questions

Other Things

- Quiz #2 next week
- Group Project Team Selection Due: tonight
- Group Project Proposal Due in two weeks
  - More on this next week
- Resume Workshop at 11AM on 3/6

Next Week: Advanced Airflow Q & A